Experiment10

1) Investigate inode structures using commands such as ls -i, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

Commands :

```
mkdir inode_lab
cd inode_lab
echo "Hello OSSP" > file1.txt
ln file1.txt hardlink.txt          # hard link
ln -s file1.txt symlink.txt       # symbolic link
ls -li
stat file1.txt
find . -inum $(stat -c %i file1.txt)
rm file1.txt
ls -li
cat hardlink.txt
cat symlink.txt
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ mkdir inode_lab
charith@DESKTOP-RBQQQSS:~$ cd inode_lab
charith@DESKTOP-RBQQQSS:~/inode_lab$ echo "Hello OSSP" > file1.txt
charith@DESKTOP-RBQQQSS:~/inode_lab$ ln file1.txt hardlink.txt
charith@DESKTOP-RBQQQSS:~/inode_lab$ ln -s file1.txt symlink.txt
charith@DESKTOP-RBQQQSS:~/inode_lab$ ls -li
total 8
639027 -rw-r--r-- 2 charith charith 11 Oct 11 08:22 file1.txt
639027 -rw-r--r-- 2 charith charith 11 Oct 11 08:22 hardlink.txt
639082 lrwxrwxrwx 1 charith charith  9 Oct 11 08:22 symlink.txt -> file1.txt
charith@DESKTOP-RBQQQSS:~/inode_lab$ stat file1.txt
  File: file1.txt
  Size: 11              Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 639027      Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/ charith)   Gid: ( 1000/ charith)
Access: 2026-10-11 08:22:00.718886085 +0000
Modify: 2026-10-11 08:22:00.724424020 +0000
Change: 2026-10-11 08:22:00.724424020 +0000
 Birth: 2026-10-11 08:22:00.718886085 +0000
charith@DESKTOP-RBQQQSS:~/inode_lab$ find . -inum $(stat -c %i file1.txt)
./hardlink.txt
./file1.txt
charith@DESKTOP-RBQQQSS:~/inode_lab$ rm file1.txt
charith@DESKTOP-RBQQQSS:~/inode_lab$ ls -li
total 4
639027 -rw-r--r-- 1 charith charith 11 Oct 11 08:22 hardlink.txt
639082 lrwxrwxrwx 1 charith charith  9 Oct 11 08:22 symlink.txt -> file1.txt
charith@DESKTOP-RBQQQSS:~/inode_lab$ cat hardlink.txt
Hello OSSP
charith@DESKTOP-RBQQQSS:~/inode_lab$ cat symlink.txt
cat: symlink.txt: No such file or directory
charith@DESKTOP-RBQQQSS:~/inode_lab$
```

2) Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/stat.h>

#define FILE_SIZE (64 * 1024 * 1024)
#define BUF_SIZE  65536

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

/* Create a test file filled with a repeating byte pattern */
void create_file(const char *name)
{
    int fd = open(name, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    unsigned char buf[BUF_SIZE];
    for (int i = 0; i < BUF_SIZE; i++) buf[i] = i % 251;
    for (long done = 0; done < FILE_SIZE; done += BUF_SIZE)
        write(fd, buf, BUF_SIZE);
    close(fd);
}

/* Read whole file with read() and compute a checksum */
unsigned long sum_read(const char *name)
{
    int fd = open(name, O_RDONLY);
    unsigned char buf[BUF_SIZE];
    unsigned long sum = 0;
    ssize_t n;
    while ((n = read(fd, buf, BUF_SIZE)) > 0)
        for (ssize_t i = 0; i < n; i++) sum += buf[i];
    close(fd);
    return sum;
}

/* Read whole file with mmap() and compute a checksum */
unsigned long sum_mmap(const char *name)
{
    int fd = open(name, O_RDONLY);
    struct stat st;
    fstat(fd, &st);
    unsigned char *p = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    if (p == MAP_FAILED) { perror("mmap"); exit(1); }
    unsigned long sum = 0;
    for (off_t i = 0; i < st.st_size; i++) sum += p[i];
    munmap(p, st.st_size);
    close(fd);
    return sum;
}

/* Copy file using read()/write() */
void copy_rw(const char *src, const char *dst)
{
    int in = open(src, O_RDONLY);
```

```
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    char buf[BUF_SIZE];
    ssize_t n;
    while ((n = read(in, buf, BUF_SIZE)) > 0)
        write(out, buf, n);
    close(in);
    close(out);
}

/* Copy file using mmap() + memcpy() */
void copy_mmap(const char *src, const char *dst)
{
    int in = open(src, O_RDONLY);
    struct stat st;
    fstat(in, &st);
    int out = open(dst, O_RDWR | O_CREAT | O_TRUNC, 0644);
    ftruncate(out, st.st_size);                    /* set size first */
    char *s = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, in, 0);
    char *d = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, out, 0);
    memcpy(d, s, st.st_size);
    msync(d, st.st_size, MS_SYNC);
    munmap(s, st.st_size);
    munmap(d, st.st_size);
    close(in);
    close(out);
}

int main(void)
{
    /* Demo: modify a file in place through a memory mapping */
    const char *demo = "mmap_demo.txt";
    int fd = open(demo, O_RDWR | O_CREAT | O_TRUNC, 0644);
    const char *text = "hello, this text will be changed using mmap.\n";
    write(fd, text, strlen(text));

    char *map = mmap(NULL, strlen(text), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) { perror("mmap"); return 1; }
    printf("Original file content : %.*s", (int)strlen(text), map);
    memcpy(map, "HELLO", 5);                       /* write through memory */
    msync(map, strlen(text), MS_SYNC);
    printf("Modified via mmap     : %.*s", (int)strlen(text), map);
    munmap(map, strlen(text));
    close(fd);

    /* Performance comparison */
    printf("\nCreating %d MB test file...\n", FILE_SIZE / (1024 * 1024));
    create_file("big.dat");

    printf("\n%-28s %-12s %s\n", "Operation", "Time (s)", "Checksum");
    printf("-------------------------------------------------------------\n");

    double t = now();
    unsigned long s1 = sum_read("big.dat");
    printf("%-28s %-12.4f %lu\n", "Read   : read()", now() - t, s1);

    t = now();
    unsigned long s2 = sum_mmap("big.dat");
    printf("%-28s %-12.4f %lu\n", "Read   : mmap()", now() - t, s2);

    t = now();
    copy_rw("big.dat", "copy_rw.dat");
    printf("%-28s %-12.4f\n", "Write  : read()/write()", now() - t);

    t = now();
    copy_mmap("big.dat", "copy_mmap.dat");
    printf("%-28s %-12.4f\n", "Write  : mmap()+memcpy()", now() - t);

    printf("\nChecksums match : %s\n", s1 == s2 ? "YES" : "NO");
    return 0;
```

```
}
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ nano exp10.2.c
charith@DESKTOP-RBQQQSS:~$ gcc exp10.2.c -o exp10.2
charith@DESKTOP-RBQQQSS:~$ ./exp10.2
Original file content : hello, this text will be changed using mmap.
Modified via mmap     : HELLO, this text will be changed using mmap.

Creating 64 MB test file...

Operation                   Time (s)     Checksum
-------------------------------------------------------------
Read   : read()             0.1523       8385715200
Read   : mmap()             0.1573       8385715200
Write  : read()/write()     0.0234
Write  : mmap()+memcpy()    0.1045

Checksums match : YES
charith@DESKTOP-RBQQQSS:~$
```